



VirtualChair — tutoriel d'exécution

1. Python:

- Version : Python 3.8 ou supérieur est recommandé.
- Téléchargement :
 - **Toutes plateformes** : Vous pouvez le télécharger depuis le site officiel : <https://www.python.org/downloads/>
 - **Linux** : Python est souvent pré-installé. Sinon, utilisez le gestionnaire de paquets de votre distribution (par exemple, `sudo apt install python3 python3-pip` sur Debian/Ubuntu, ou `sudo yum install python3 python3-pip` sur Fedora).
 - **macOS** : Vous pouvez l'installer via le site officiel ou via Homebrew (`brew install python`).
- Configuration du PATH :
 - **Windows** : Lors de l'installation depuis le site officiel, cochez la case "Add Python to PATH" (ou "Ajouter Python au PATH").
 - **macOS/Linux** : Les installations via gestionnaire de paquets ou Homebrew configurent généralement le PATH automatiquement.
- Vérification : Ouvrez un terminal (Invite de commandes ou PowerShell sur Windows, Terminal sur macOS/Linux) et tapez `python3 --version` (ou `python --version` sur certaines configurations Windows après installation manuelle).

2. pip (Gestionnaire de paquets Python):

- Normalement, pip est installé automatiquement avec Python 3.4+
- Vérification : Dans votre terminal, tapez `pip3 --version` ou `pip --version` .

- Si pip n'est pas installé (rare), suivez les instructions sur <https://pip.pypa.io/en/stable/installation/>.

Étape 1: Configuration de l'environnement

1. Clonez le projet (si ce n'est pas déjà fait):

Assurez-vous d'avoir les fichiers du projet virtualchair sur votre machine.

2. Naviguez dans le répertoire du projet: Bash

Ouvrez un terminal et déplacez-vous dans le dossier où vous avez cloné ou téléchargé le projet.

```
cd chemin/vers/virtualchair
```

3. Créez un environnement virtuel: Bash

Un environnement virtuel isole les dépendances de votre projet.

```
python3 -m venv venv
```

Si `python3` n'est pas reconnu, essayez avec `python`.

4. Activez l'environnement virtuel:

La commande varie selon votre système d'exploitation et votre shell :

- **Windows:**

- Invite de commandes (cmd.exe):

DOS

```
venv\Scripts\activate
```

- PowerShell:

PowerShell

(Si vous rencontrez une erreur d'exécution de script dans PowerShell, vous devrez peut-être exécuter

```
Set-ExecutionPolicy Unrestricted -Scope Process
```

 puis réessayer.)

```
.\venv\Scripts\Activate.ps1
```

- **macOS / Linux** (bash/zsh): Bash

```
source venv/bin/activate
```

Une fois l'environnement activé, vous devriez voir `(venv)` au début de l'invite de votre terminal.

Étape 2: Installation des dépendances

1. Installez les paquets requis: Bash

Utilisez pip pour installer les dépendances listées dans le fichier requirements.txt. Assurez-vous que votre environnement virtuel est activé.

```
pip install -r requirements.txt
```

Cela installera toutes les bibliothèques nécessaires, telles que Flask, SQLAlchemy, etc.

2. Configurez les variables d'environnement:

- Créez un fichier nommé `.env` à la racine du projet (au même niveau que `requirements.txt`). Vous pouvez copier le contenu du fichier `.env.example` s'il existe et le renommer en `.env`.

- Remplissez les informations nécessaires. Voici un exemple de structure de fichier `.env`

```
DB_USER=postgres
DB_PASSWORD=votre_mot_de_passe_bdd
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mydatabase

AWS_ACCESS_KEY_ID=VOTRE_CLE_AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY=VOTRE_CLE_AWS_SECRET_ACCESS_KEY
AWS_REGION=eu-west-3 # ou votre région S3
S3_BUCKET_NAME=nom_de_votre_bucket_s3

DEBUG=True
SECRET_KEY=une_cle_secrete_aleatoire_et_forte
```

- Remplacez les valeurs d'exemple (`votre_mot_de_passe_bdd` , `VOTRE_CLE_AWS...` , `nom_de_votre_bucket_s3` , `une_cle_secrete_aleatoire_et_forte`) par vos propres configurations.
- La `SECRET_KEY` de Flask est cruciale pour la sécurité des sessions. Vous pouvez générer une clé forte avec Python :Python

```
import secrets
print(secrets.token_hex(24))
```
- Contactez @Souleimane E. si des informations spécifiques au projet (comme les identifiants AWS de développement ou la configuration de la base de données partagée) doivent vous être transmises.

Étape 3: Lancement de l'application

1. Assurez-vous d'être dans le bon répertoire:

Votre terminal doit être dans le répertoire racine du projet virtualchair (celui qui contient le dossier venv et le fichier .env). L'environnement virtuel (venv) doit être activé.

2. Lancez l'application Flask:Bash

La commande exacte peut dépendre de la structure de votre projet virtualchair. Si le point d'entrée de l'application est virtualchair/core/app.py, la commande suivante, exécutée depuis la racine du projet, devrait fonctionner :

```
python3 -m virtualchair.core.app
```

Si `python3` n'est pas reconnu, essayez `python` .

3. Vérifiez la sortie du serveur:

Vous devriez voir des messages dans le terminal indiquant que le serveur Flask est en cours d'exécution. Par défaut, il est souvent accessible à `http://127.0.0.1:8080/` ou `http://0.0.0.0:8080/`. Le message du terminal précisera l'adresse.

4. Accédez à l'application dans votre navigateur:

Ouvrez votre navigateur web et accédez à l'adresse indiquée par le serveur (par exemple, `http://127.0.0.1:8080/` ou `http://localhost:8080/`). Vous devriez voir l'interface de VirtualChair.

Dépannage

- **Erreurs d'installation des paquets (`pip install`):**

- Assurez-vous que votre environnement virtuel est activé.
- Mettez à jour pip : `pip install --upgrade pip` .
- Sur Linux, certains paquets peuvent nécessiter des outils de compilation ou des bibliothèques de développement (par exemple, `python3-dev` , `build-essential`). Lisez attentivement les messages d'erreur, ils indiquent souvent les dépendances manquantes. Pour `psycopg2-binary` , cela est généralement moins un problème, mais si vous utilisiez `psycopg2` (la version source), vous

auriez besoin des outils de développement PostgreSQL (`libpq-dev` sur Debian/Ubuntu).

- **Erreurs de connexion à la base de données:**

- Vérifiez que votre serveur PostgreSQL est en cours d'exécution.
- Assurez-vous que les informations d'identification (utilisateur, mot de passe, hôte, port, nom de la base) dans votre fichier `.env` sont correctes et correspondent à votre configuration PostgreSQL.
- Vérifiez que la base de données spécifiée (`DB_NAME`) existe sur votre serveur PostgreSQL.
- Confirmez que le paquet `psycopg2-binary` (ou `psycopg2`) est bien installé dans votre environnement virtuel (`pip list | grep psycopg2`).

- **Problèmes d'exécution de l'application:**

- Vérifiez attentivement les messages d'erreur affichés dans le terminal lors du lancement.
- Assurez-vous que toutes les dépendances sont correctement installées et que toutes les variables d'environnement nécessaires dans `.env` sont configurées.
- Si vous avez modifié du code, vérifiez la syntaxe et les importations.

- **Commande `python` vs `python3` :**

- Sur de nombreux systèmes Linux et macOS récents, `python` peut encore pointer vers Python 2. Il est plus sûr d'utiliser `python3` et `pip3` explicitement. Si `python3` n'est pas trouvé, essayez `python` (cela peut signifier que votre système a `python` aliasé sur Python 3, ou que seul Python 3 est installé sous ce nom).

- **Variables d'environnement non chargées:**

- Assurez-vous que le fichier `.env` est bien à la racine de votre projet et qu'il est correctement nommé (commence par un point, pas d'autre extension).
- L'application doit utiliser une bibliothèque comme `python-dotenv` (qui serait listée dans `requirements.txt`) pour charger automatiquement ce fichier. Vérifiez

que la ligne `load_dotenv()` est présente au début du script de lancement de votre application.
